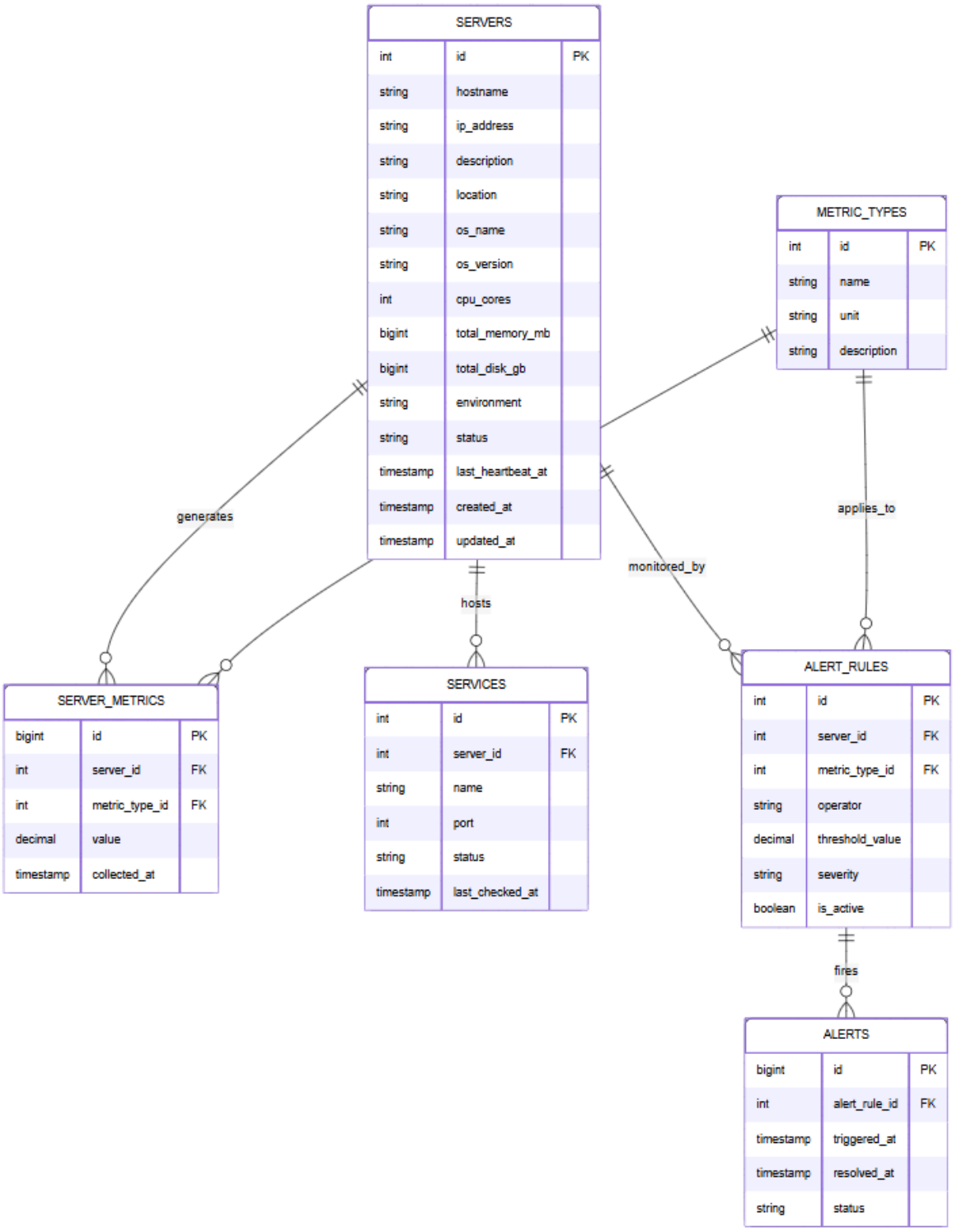
PART 1: Web Development & Automation

1.

a. See DER



b. Project Repository

The complete implementation of the monitoring system, including the PHP application, Docker environment, database integration, monitoring scripts, and supporting documentation, can be found in the [monitoring-system-v2]() repository.

Please refer to the repository for the full source code, configuration files, deployment instructions, and implementation details discussed throughout this document.

c. Server Monitoring Process (See QUESTION_1_C.md in repository)

Overview

To monitor the infrastructure and provide data to the web application, I propose an agent-based monitoring architecture. A monitoring agent periodically collects information from the servers, stores the results in the database, and the web application consumes this data to display dashboards, metrics, alerts, and server status information.

The monitoring process is executed on a schedule (e.g., every 5 minutes using cron) and consists of the following steps:

1. Check if each server is up and running
2. Retrieve the list of monitored servers
3. Verify if the web server process is running
4. Collect CPU and memory usage
5. Store collected information in the database
6. Evaluate alert rules and generate alerts when necessary

Proposed Architecture

```text
+---------------------+
| Monitored Servers   |
+---------------------+
        ^
        |
        | SSH
        |
+---------------------+
| Monitoring Agent    |
| PHP CLI + Cron      |
+---------------------+
        |
        | INSERT / UPDATE
        v
+---------------------+
| MySQL Database      |
+---------------------+
        |
```

```
        | REST API
        v
+--------------------+
| Web Dashboard      |
+--------------------+
```

Architecture Rationale

This architecture separates monitoring responsibilities from presentation responsibilities.

Benefits:

* Scalability: new servers can be added through the database without changing the monitoring code.
* Decoupling: monitoring continues even if the web application is temporarily unavailable.
* Historical analysis: metrics are stored as time-series data and can be visualized later.
* Extensibility: new metrics can be added without modifying the database schema.
* Simplicity: SSH is already available in most Linux environments and does not require installing additional monitoring software on target servers.

Remote Command Execution

To collect information from remote servers, the monitoring agent uses SSH connections.

A dedicated monitoring user (e.g., `monitor`) is configured on every monitored server with SSH key-based authentication. This allows the monitoring agent to securely execute commands without storing passwords or requiring manual intervention.

Architecture:

```text
+---------------------+
| Monitoring Agent    |
| (PHP CLI Script)    |
+---------------------+
         |
         | SSH
         v
+---------------------+
| Linux Server        |
+---------------------+
         |
         +--> ping
         +--> systemctl is-active nginx
         +--> free -m
```

```
        +--> top -bn1
```

The monitoring process retrieves the server IP address from the database and executes remote commands through SSH.

Example:

```bash
ssh monitor@10.0.0.10 "free -m"
```

CPU usage:

```bash
ssh monitor@10.0.0.10 "top -bn1"
```

Web server status:

```bash
ssh monitor@10.0.0.10 "systemctl is-active nginx"
```

Example PHP implementation:

```php
$command = sprintf(
    'ssh monitor@%s "free -m"',
    $serverIp
);

$output = shell_exec($command);
```

This approach was chosen because it is simple, secure, and commonly used in Linux environments. It does not require additional software installation on monitored servers and can easily scale by adding new server records to the database.

Step 1 - Check if Each Server is Up and Running

The monitoring agent retrieves all active servers from the database.

```sql
SELECT id, hostname, ip_address
FROM servers
```

WHERE status = 'active';
```

For each server, a connectivity test is executed.

Example Linux command:

```bash
ping -c 3 10.0.0.10
```

Example PHP code:

```php
$output = shell_exec(
    "ping -c 3 {$ipAddress}"
);
```

Success Scenario

If the server responds:

```text
Server Status = UP
```

The heartbeat timestamp is updated:

```sql
UPDATE servers
SET last_heartbeat_at = NOW()
WHERE id = ?;
```

Failure Scenario

If the server does not respond:

```text
Server Status = DOWN
```

An alert can be generated to notify administrators.

Example:

```sql

INSERT INTO alerts (...)
```

The alert will appear in the monitoring dashboard.

Step 2 - Retrieve the List of Servers to Monitor

The monitoring process retrieves servers from the `servers` table.

Example:

```sql
SELECT *
FROM servers
WHERE status = 'active';
```

Only active production servers may be monitored.

Example:

```sql
SELECT *
FROM servers
WHERE
    status = 'active'
    AND environment = 'production';
```

This approach allows administrators to control monitoring behavior directly from the database.

The monitoring agent executes this query at the beginning of every monitoring cycle.

Step 3 - Verify if the Web Server Process is Running

Each server may have one or more monitored services registered in the `services` table.

Example:

Server	Service
web01	nginx
web01	php-fpm
```

| api01 | apache2 |

The monitoring agent verifies whether each service is running.

Example Linux command:

```bash
systemctl is-active nginx
```

Alternative:

```bash
ps aux | grep nginx
```

Executed remotely via SSH:

```bash
ssh monitor@10.0.0.10 "systemctl is-active nginx"
```

Example PHP:

```php
$status = trim(
    shell_exec(
        "ssh monitor@{$serverIp} 'systemctl is-active nginx'"
    )
);
```

Possible results:

```text
active
inactive
failed
```

The result is stored in the database:

```sql
UPDATE services
SET
    status = 'running',
    last_checked_at = NOW()
WHERE id = ?;
```

```

If a critical service is stopped, an alert should be created.

Example alert:

```text
Critical: nginx process stopped on web01
```

Step 4 - Collect CPU and Memory Usage

The monitoring agent collects performance information remotely through SSH.

CPU Usage

Example command:

```bash
ssh monitor@10.0.0.10 "top -bn1"
```

Alternative:

```bash
ssh monitor@10.0.0.10 "mpstat"
```

Memory Usage

Example command:

```bash
ssh monitor@10.0.0.10 "free -m"
```

Example result:

```text
CPU Usage: 64.5%
Memory Usage: 72.0%
```

The values are stored in the `server_metrics` table.

CPU:
```

```sql
INSERT INTO server_metrics
(
    server_id,
    metric_type_id,
    value,
    collected_at
)
VALUES
(
    1,
    1,
    64.5,
    NOW()
);
```

Memory:

```sql
INSERT INTO server_metrics
(
    server_id,
    metric_type_id,
    value,
    collected_at
)
VALUES
(
    1,
    2,
    72.0,
    NOW()
);
```

Where:

Metric Type	Description
1	CPU Usage
2	Memory Usage

This structure allows storing historical measurements for chart generation and trend analysis.

Alert Evaluation

After metrics are collected, the monitoring agent evaluates the configured alert rules.

Example rule:

```text
CPU Usage > 80%
Severity = Critical
```

Stored in:

```sql
alert_rules
```

Query:

```sql
SELECT *
FROM alert_rules
WHERE
    metric_type_id = 1
    AND is_active = TRUE;
```

If the collected metric exceeds the threshold:

```text
CPU Usage = 85%
Threshold = 80%
```

A new alert is created:

```sql
INSERT INTO alerts
(
    alert_rule_id,
    server_id,
    message
)
VALUES
(
    5,
    1,
```

```
    'CPU usage exceeded threshold'
);
```

The alert will then be displayed in the web dashboard.

Database Usage

The monitoring process interacts with the following tables:

Table	Purpose
servers	Stores server information and heartbeat
services	Stores monitored services and their status
metric_types	Defines available metric categories
server_metrics	Stores collected metrics over time
alert_rules	Defines alert thresholds
alerts	Stores generated alerts

This structure provides complete traceability of monitoring information.

Additional Information Worth Monitoring

Besides availability, CPU, and memory usage, I would also monitor:

Disk Usage

Command:

```bash
ssh monitor@10.0.0.10 "df -h"
```

Reason:

High disk utilization is one of the most common causes of production incidents.

Disk I/O

Command:

```bash
```

ssh monitor@10.0.0.10 "iostat"
```

Reason:

Helps identify storage bottlenecks.

---

## Network Traffic

Command:

```bash
ssh monitor@10.0.0.10 "sar -n DEV"
```

Reason:

Detects abnormal network activity and bandwidth saturation.

System Load Average

Command:

```bash
ssh monitor@10.0.0.10 "uptime"
```

Reason:

Provides an overall view of server load.

SSL Certificate Expiration

Reason:

Prevents service outages caused by expired certificates.

Application Response Time

Command:
```

```bash
curl -o /dev/null -s -w "%{time_total}" https://application-url
```

Reason:

Measures application performance from an end-user perspective.

Process Restart Count

Reason:

Identifies unstable services that frequently crash and restart.

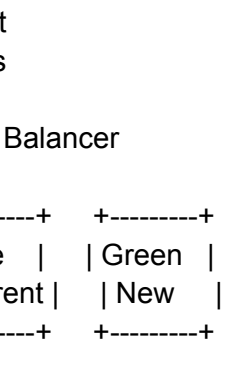
Monitoring Flow Summary

```text
1. Load active servers from database
        |
        v
2. Connect to server via SSH
        |
        v
3. Check connectivity
        |
        v
4. Update heartbeat
        |
        v
5. Check monitored services
        |
        v
6. Update services status
        |
        v
7. Collect CPU and memory metrics
        |
        v
8. Store metrics in server_metrics
        |
        v
9. Evaluate alert rules
        |
```

v
10. Generate alerts if necessary
|
v
11. Display information in the web dashboard
```

## Conclusion

The proposed solution uses a simple and scalable monitoring architecture based on a PHP CLI monitoring agent executed by cron jobs. The agent connects to remote servers using SSH, collects availability, service, CPU, and memory information, and stores the results in a centralized MySQL database.

The web dashboard consumes this information to provide historical metrics, service status, server health information, and alert management. This architecture is easy to implement, requires minimal infrastructure, fully leverages the proposed database model, and can be extended with additional metrics such as disk usage, network traffic, and application response time.

## 2. Software Upgrade Plan for a Critical System

When upgrading a critical production system, my main goal would be to minimize downtime while ensuring that any issue can be quickly reverted without impacting customers.

## Planning and Validation

Before performing the upgrade, I would review the release notes, identify any breaking changes, and understand whether the new version introduces infrastructure, database, or integration changes.

I would then execute the entire upgrade process in a staging environment that mirrors production. This allows me to validate critical functionality such as authentication, APIs, background jobs, and database operations before touching production.

## Backup and Rollback Strategy

Before deployment, I would create and validate backups of:

* Database
* Application files
* Configuration files

At the same time, I would prepare a rollback plan. If the new version introduces critical issues, performance degradation, or business-impacting bugs, the previous version must be restorable within minutes.
```

Blue-Green Deployment

To reduce downtime, I would use a Blue-Green deployment strategy.



The new version would be deployed to the Green environment and fully validated before receiving traffic. Once validated, traffic would be switched from Blue to Green. If problems occur, traffic can immediately be redirected back to Blue.

This approach provides near-zero downtime and a very fast rollback mechanism.

Controlled Rollout with Feature Flags

Even after deploying the new version, I would not expose it to all customers immediately.

Instead, I would use feature flags to control which customers access the new functionality.

Example:

Customer	Version
Client A	v1
Client B	v1
Client C	v2

The rollout would happen gradually:

1. Internal users and QA team
2. Small pilot group of customers
3. Partial rollout
4. Full rollout

This reduces risk and allows monitoring of real-world behavior before enabling the new version for everyone.

Backward-Compatible Data Strategy

One of the biggest challenges during a migration is data compatibility.

If Version 2 introduces a new data model, customers using Version 1 must still be able to operate normally in case a rollback is required.

For this reason, I would implement a translation layer that converts data from the new model into a format compatible with the previous version before persisting it.

In more complex scenarios, I would adopt a dual-write strategy, where Version 2 stores both:

* The new data structure
* A compatible legacy representation

This ensures that rolling back to Version 1 does not require emergency database conversions or data recovery procedures.

Monitoring and Post-Deployment Validation

After deployment, I would continuously monitor:

* Error rates
* Response times
* CPU and memory usage
* Application logs
* Business KPIs

Additionally, smoke tests would be executed to validate critical business workflows and integrations.

Only after observing stable behavior would I continue expanding the rollout to additional customers.

Conclusion

My upgrade strategy combines staging validation, tested backups, rollback planning, Blue-Green deployment, customer-based feature flags, and backward-compatible data persistence.

This approach minimizes downtime, reduces operational risk, enables rapid rollback, and ensures that customers can be migrated gradually while maintaining compatibility between application versions.

3. Deployment Process for a Python CLI Application

Since the application is a Python CLI tool used by hundreds of employees across multiple teams, my focus would be on making installation, updates, and version management as simple as possible.

I would package the application as a Python package and publish it to an internal package repository (similar to a private PyPI). This ensures that all users install approved versions of the tool and that releases are centralized.

The deployment process would be automated through a CI/CD pipeline. Whenever a new version is released, the pipeline would:

1. Run automated tests
2. Build the package
3. Publish the package to the internal repository
4. Create a new versioned release

To avoid dependency conflicts on users' machines, I would distribute the application using `pipx`, which installs CLI applications in isolated environments.

Example installation:

```bash
pipx install my-cli
```

For versioning, I would follow Semantic Versioning:

```text
1.0.0 -> Initial release
1.1.0 -> New features
1.1.1 -> Bug fixes
2.0.0 -> Breaking changes
```

To simplify upgrades, users could update the application through a command such as:

```bash
pipx upgrade my-cli
```

If a problem is discovered after a release, rollback is straightforward because previous versions remain available in the repository:

```bash
pipx install my-cli==1.1.0
```

This approach provides a standardized installation process, centralized version control, simple upgrades, and fast rollback capabilities, making it suitable for a CLI application used continuously by hundreds of users across the company.

4. When Do I Decide to Automate a Task?

I do not follow a strict rule, but if I have to perform the same task three or more times, I usually start thinking about automation.

My decision depends on factors such as how often the task is executed, how much time it takes, and the risk of human error. If a repetitive task can be automated with reasonable effort and will save time in the future, I believe automation is worthwhile.

Beyond efficiency, automation also improves consistency and reliability by reducing manual mistakes and ensuring the task is executed the same way every time.

For me, the goal of automation is not only to save time, but also to allow people to focus on more valuable and less repetitive work.

5. Python project

Note: The implementation related to this question can be found in the python directory of the monitoring-system-v2 repository.

6. Investigating the Runtime Degradation

My first step would be to identify where the additional time is being spent.

Since the system is composed of a wrapper and two internal modules, I would measure the execution time of each component separately to determine whether the slowdown comes from Module A, Module B, or the wrapper itself.

After identifying the affected component, I would compare what changed between June and July, such as:

* New deployments or code changes
* Configuration changes
* Increased data volume
* Higher system usage
* Infrastructure changes

I would also review logs and monitor metrics such as CPU, memory, database performance, and external API response times to identify any bottlenecks.

In summary, my approach would be:

1. Measure the runtime of each module independently.
2. Identify which component is causing the degradation.
3. Compare changes made before the issue appeared.
4. Analyze logs and infrastructure metrics.

5. Determine the root cause and implement corrective actions.

This approach helps isolate the problem quickly and focus the investigation on the component responsible for the increased runtime.